

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**
Факультет информационных технологий
Кафедра параллельных вычислений

ОТЧЕТ
О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ

«MPI+POSIX Threads»

студента 2 курса, 24203 группы

Анисимова Льва Евгеньевича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
доцент кафедры параллельных
вычислений
Андрей Юрьевич Власенко

Новосибирск, 2026

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
Приложение 1. <i>Код программы tri.spp</i>	7
Приложение 2. <i>Код скрипта запуска на кластере</i>	13
Приложение 3. <i>Таблицы</i>	15
Приложение 4. <i>Графики</i>	16
Приложение 5. <i>Профилирование</i>	17

ЦЕЛЬ

Освоить разработку многопоточных программ с использованием POSIX Threads API. Познакомиться с задачей динамического распределения работы между процессорами.

ЗАДАНИЕ

1. Написать параллельную программу на языке C/C++ с использованием MPI и POSIX Threads, реализующую алгоритм динамической балансировки нагрузки (Work Stealing) для обработки списков независимых задач с неизвестным вычислительным весом. Обеспечить многопоточную архитектуру внутри каждого MPI-процесса (минимум две нити: вычислитель и коммуникатор) и безопасный доступ к данным через мьютексы. Проверить корректность распределения задач при использовании детерминированной функции веса, имитирующей дисбаланс, и убедиться в достижении глобальной синхронизации после каждой итерации.
2. Измерить время работы программы при использовании различного числа процессорных ядер: 1, 2, 4, 8, 16. Параметры задачи (коэффициент веса L и количество итераций) подобрать таким образом, чтобы решение на одном ядре занимало не менее 30 секунд. По результатам измерений рассчитать показатели дисбаланса (Δk в секундах и процентах), а также построить графики зависимости времени выполнения, ускорения и эффективности распараллеливания от числа используемых ядер.
3. Произвести профилирование программы (с использованием инструментов профилировки MPI) и выполнить оптимизацию механизмов взаимодействия потоков или параметров опроса сети.
4. Составить отчет.

ОПИСАНИЕ РАБОТЫ

В данной лабораторной работе реализована параллельная программа для итеративной обработки независимых задач с использованием гибридной модели параллелизма MPI + POSIX Threads. Основной целью работы является минимизация времени простоя процессов при выполнении задач с заранее неизвестным и неравномерным вычислительным весом.

Для эффективного распределения ресурсов применена стратегия динамической балансировки нагрузки. В рамках каждого MPI-процесса порождаются две нити, работающие в общем адресном пространстве: `worker_thread` (вычислитель) и `listener_thread` (коммуникатор).

В исходном коде применены следующие особенности:

1. Асинхронное обслуживание запросов: Коммуникационная нить `listener` работает в режиме непрерывного опроса сети с использованием неблокирующей функции `MPI_Iprobe`. Это позволяет процессу отвечать на запросы о «краже» задач от других узлов в реальном времени, даже когда вычислительная нить полностью загружена расчетами. Использование `MPI_Iprobe` вместо блокирующего ожидания исключает возникновение дедлока при встречных запросах.

2. Протокол кражи задач: Когда у процесса заканчиваются локальные задания, вычислительная нить переходит в режим ожидания, а коммуникационная нить начинает опрашивать соседние процессы. В случае успеха донор передает ровно 50% своих текущих невыполненных задач. Решение о передаче принимается исключительно на основе количества задач в очереди, без анализа их вычислительной сложности, что имитирует работу с неизвестными весами.

3. Синхронизация через примитивы POSIX: Доступ к общей очереди задач внутри процесса защищен с помощью мьютекса `pthread_mutex_t`. Это гарантирует отсутствие состояния гонки (`race condition`) в моменты, когда `worker` извлекает задачу для выполнения, а `listener` в это же время пытается отдать часть задач внешнему процессу. Для возобновления работы вычислителя после кражи новой порции данных используется механизм условных переменных `pthread_cond_t`.

4. Централизованное управление завершением: Координация окончания итерации возложена на процесс с рангом 0. Процессы, исчерпавшие свои задачи и не нашедшие их у соседей, отправляют мастеру отчет о простое TAG_STOP_REPORT. Если процесс возобновляет работу, он сообщает об этом через TAG_BUSY_REPORT. Итерация считается завершенной только тогда, когда ранг 0 зафиксирует простой всех процессов в системе, после чего рассылается широковещательный сигнал TAG_FINAL_STOP.

5. Безопасность потоков: Программа инициализируется с уровнем поддержки MPI_THREAD_MULTIPLE, что позволяет нескольким потокам одновременно вызывать функции библиотеки MPI, обеспечивая максимальную независимость вычислительной и сетевой логики.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно разработана и протестирована гибридная параллельная реализация динамической балансировки нагрузки на базе MPI и POSIX Threads.

Алгоритм продемонстрировал исключительную эффективность выравнивания загрузки: во всех тестовых запусках (от 2 до 24 процессов) показатель дисбаланса не превысил 0.03%, а разница между временем работы самого быстрого и медленного процессов (Δ) составила сотые доли секунды. Это доказывает корректность логики протокола кражи задач и надежность реализованной схемы синхронизации между вычислительной и коммуникационной нитями через мьютексы и условные переменные.

Анализ метрик производительности выявил следующие особенности работы системы:

- **Идеальное распределение:** Механизм кражи задач полностью нивелирует искусственно созданный дисбаланс весов, обеспечивая равномерную загрузку ядер на протяжении всех итераций.
- **Масштабируемость и накладные расходы:** Несмотря на идеальную балансировку, общее время выполнения T_{total} растет пропорционально числу процессов (с 10.1 с на 1 ядре до 165.3 с на 24 ядрах). Это привело к снижению эффективности E до минимальных значений при $NP = 24$.

Столь значительное падение производительности при увеличении числа узлов объясняется спецификой задачи: накладные расходы на поддержание работы нити-слушателя, частые вызовы неблокирующих проверок MPI_Iprobe и конкуренцию потоков за мьютекс при обращении к очереди задач. Для данного объема вычислений (параметр L) стоимость межпроцессорных взаимодействий и синхронизации внутри гибридной модели превысила выигрыш от параллельного исполнения. Тем не менее, архитектурная цель работы достигнута: создана отказоустойчивая система, способная динамически перераспределять нагрузку в распределенной среде без участия центрального диспетчера.

Приложение 1. Код программы *tri.crr*

```
#include <mpi.h>
#include <pthread.h>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <unistd.h>

static constexpr int TASKS_PER_PROC = 200;
static constexpr int L = 10000;
static constexpr int NUM_ITER = 10;

enum Tags {
    TAG_REQUEST = 1, TAG_REPLY, TAG_DATA,
    TAG_STOP_REPORT, TAG_BUSY_REPORT, TAG_FINAL_STOP
};

typedef struct { int id; int repeatNum; } Task;

typedef struct {
    int rank, size;
    Task* tasks;
    int taskCount;
    int front;
    pthread_mutex_t mu;
    double globalRes;
    int doneTasks;
    int workerIdle;
    int iterDone;
    int reported_idle;
    int idle_count;
    pthread_cond_t cvWorker;
} State;

typedef struct { State* st; int iter; } Args;

static int task_weight(int i, int iter, int size) {
    int a = abs(50 - i % 100);
    int b = abs((i / 100) - (iter % size));
    return (a + 1) * (b + 1) * L;
}

static void* worker_thread(void* arg) {
    State* st = ((Args*)arg)->st;
    while (true) {
        pthread_mutex_lock(&st->mu);
        while (st->front >= st->taskCount && !st->iterDone)
            pthread_cond_wait(&st->cvWorker, &st->mu);
        if (st->iterDone && st->front >= st->taskCount) {
            pthread_mutex_unlock(&st->mu);
            break;
        }
        Task t = st->tasks[st->front++];
        pthread_mutex_unlock(&st->mu);

        double res = 0.0;
```

```

        for (int k = 0; k < t.repeatNum; k++) res += sin(t.id + k);

        pthread_mutex_lock(&st->mu);
        st->globalRes += res;
        st->doneTasks++;
        if (st->front >= st->taskCount) st->workerIdle = 1;
        pthread_mutex_unlock(&st->mu);
    }
    return NULL;
}

static void handle_requests(State* st) {
    int flag; MPI_Status status;
    while (MPI_Iprobe(MPI_ANY_SOURCE, TAG_REQUEST, MPI_COMM_WORLD,
&flag, &status), flag) {
        int src = status.MPI_SOURCE;
        int dummy; MPI_Recv(&dummy, 1, MPI_INT, src, TAG_REQUEST,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        pthread_mutex_lock(&st->mu);
        int give = (st->taskCount - st->front) / 2;
        if (give < 1) give = 0;
        MPI_Send(&give, 1, MPI_INT, src, TAG_REPLY, MPI_COMM_WORLD);
        if (give > 0) {
            MPI_Send(st->tasks + (st->taskCount - give), give *
(int)sizeof(Task), MPI_BYTE, src, TAG_DATA, MPI_COMM_WORLD);
            st->taskCount -= give;
        }
        pthread_mutex_unlock(&st->mu);
    }
    if (st->rank == 0) {
        while (MPI_Iprobe(MPI_ANY_SOURCE, TAG_STOP_REPORT,
MPI_COMM_WORLD, &flag, &status), flag) {
            int d; MPI_Recv(&d, 1, MPI_INT, status.MPI_SOURCE,
TAG_STOP_REPORT, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            st->idle_count++;
        }
        while (MPI_Iprobe(MPI_ANY_SOURCE, TAG_BUSY_REPORT,
MPI_COMM_WORLD, &flag, &status), flag) {
            int d; MPI_Recv(&d, 1, MPI_INT, status.MPI_SOURCE,
TAG_BUSY_REPORT, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            st->idle_count--;
        }
    } else if (MPI_Iprobe(0, TAG_FINAL_STOP, MPI_COMM_WORLD, &flag,
&status), flag) {
        int d; MPI_Recv(&d, 1, MPI_INT, 0, TAG_FINAL_STOP,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        pthread_mutex_lock(&st->mu);
        st->iterDone = 1;
        pthread_cond_signal(&st->cvWorker);
        pthread_mutex_unlock(&st->mu);
    }
}

static void* listener_thread(void* arg) {
    State* st = ((Args*)arg)->st;
    while (1) {
        handle_requests(st);
        pthread_mutex_lock(&st->mu);
    }
}

```



```

        if (st->iterDone) { pthread_mutex_unlock(&st->mu); break; }
        int idle = st->workerIdle;
        pthread_mutex_unlock(&st->mu);

        if (idle) {
            int success = 0;
            for (int i = 1; i < st->size && !success; i++) {
                int target = (st->rank + i) % st->size;
                int d = 0; MPI_Send(&d, 1, MPI_INT, target,
TAG_REQUEST, MPI_COMM_WORLD);
                int count = -1;
                while (1) {
                    handle_requests(st);
                    int f; MPI_Status s;
                    MPI_Iprobe(target, TAG_REPLY, MPI_COMM_WORLD, &f,
&s);
                    if (f) {
                        MPI_Recv(&count, 1, MPI_INT, target,
TAG_REPLY, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                        break;
                    }
                    if (st->iterDone) break;
                    usleep(500);
                }
                if (count > 0) {
                    Task* buf = (Task*)malloc((size_t)count *
sizeof(Task));
                    MPI_Recv(buf, count * (int)sizeof(Task), MPI_BYTE,
target, TAG_DATA, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                    pthread_mutex_lock(&st->mu);
                    if (st->reported_idle && st->rank != 0) {
                        int b = 1; MPI_Send(&b, 1, MPI_INT, 0,
TAG_BUSY_REPORT, MPI_COMM_WORLD);
                        st->reported_idle = 0;
                    }
                    st->tasks = (Task*)realloc(st->tasks, (size_t)(st-
>taskCount + count) * sizeof(Task));
                    memcpy(st->tasks + st->taskCount, buf,
(size_t)count * sizeof(Task));
                    st->taskCount += count; st->workerIdle = 0;
                    pthread_cond_signal(&st->cvWorker);
                    pthread_mutex_unlock(&st->mu);
                    free(buf); success = 1;
                }
                if (st->iterDone) break;
            }
            if (!success && !st->iterDone) {
                if (st->rank != 0) {
                    if (!st->reported_idle) {
                        int s = 1; MPI_Send(&s, 1, MPI_INT, 0,
TAG_STOP_REPORT, MPI_COMM_WORLD);
                        st->reported_idle = 1;
                    }
                } else {
                    pthread_mutex_lock(&st->mu);
                    if (st->idle_count == st->size - 1) {
                        st->iterDone = 1; pthread_cond_signal(&st-
>cvWorker);

```

```

        for (int k = 1; k < st->size; k++) {
            int stop = 1; MPI_Send(&stop, 1, MPI_INT,
k, TAG_FINAL_STOP, MPI_COMM_WORLD);
        }
        pthread_mutex_unlock(&st->mu);
    }
    }
    usleep(1000);
}
return NULL;
}

int main(int argc, char** argv) {
    int prov; MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE,
&prov);
    State st; memset(&st, 0, sizeof(st));
    MPI_Comm_rank(MPI_COMM_WORLD, &st.rank);
    MPI_Comm_size(MPI_COMM_WORLD, &st.size);
    pthread_mutex_init(&st.mu, NULL);
    pthread_cond_init(&st.cvWorker, NULL);

    if (st.rank == 0)
        printf("=== MPI+threads | procs=%d tasks/proc=%d L=%d
iters=%d ===\n\n",
            st.size, TASKS_PER_PROC, L, NUM_ITER);

    double* allTimes = (st.rank == 0)
        ? (double*)malloc((size_t)st.size *
sizeof(double))
        : NULL;

    for (int iter = 0; iter < NUM_ITER; iter++) {
        pthread_mutex_lock(&st.mu);
        if (st.tasks) free(st.tasks);
        st.tasks = (Task*)malloc((size_t)TASKS_PER_PROC *
sizeof(Task));
        st.taskCount = TASKS_PER_PROC; st.front = 0; st.globalRes =
0.0; st.doneTasks = 0;
        st.workerIdle = 0; st.iterDone = 0; st.reported_idle = 0;
st.idle_count = 0;
        for (int i = 0; i < TASKS_PER_PROC; i++) {
            int gid = st.rank * TASKS_PER_PROC + i;
            st.tasks[i].id = gid;
            st.tasks[i].repeatNum = task_weight(gid, iter, st.size);
        }
        pthread_mutex_unlock(&st.mu);

        Args args = {&st, iter};
        pthread_t wt, lt;
        pthread_create(&wt, NULL, worker_thread, &args);
        pthread_create(&lt, NULL, listener_thread, &args);

        double t1 = MPI_Wtime();
        pthread_join(lt, NULL);
        pthread_join(wt, NULL);
        double t2 = MPI_Wtime();

```

```

double localTime = t2 - t1;

printf("  [rank %d] iter=%d  t=%.4f s  tasks=%d  res=%.6f\n",
      st.rank, iter, localTime, st.doneTasks, st.globalRes);
fflush(stdout);

MPI_Gather(&localTime, 1, MPI_DOUBLE,
          allTimes, 1, MPI_DOUBLE,
          0, MPI_COMM_WORLD);

if (st.rank == 0) {
    double tmax = allTimes[0], tmin = allTimes[0];
    for (int p = 1; p < st.size; p++) {
        if (allTimes[p] > tmax) tmax = allTimes[p];
        if (allTimes[p] < tmin) tmin = allTimes[p];
    }
    double imb = (tmax > 0.0) ? (tmax - tmin) / tmax * 100.0 :
0.0;
    printf("  [iter %d] Tmax=%.4f s  Tmin=%.4f s  delta=%.4f s
imbalance=%.2f%%\n\n",
          iter, tmax, tmin, tmax - tmin, imb);
    fflush(stdout);
}

if (st.rank == 0) free(allTimes);
free(st.tasks);
pthread_mutex_destroy(&st.mu);
pthread_cond_destroy(&st.cvWorker);
MPI_Finalize();
return 0;
}

```

Приложение 2. Код скрипта запуска на кластере

```
#!/bin/bash
#PBS -N Lab6_MPI_pthreads
#PBS -l nodes=2:ppn=12
#PBS -l walltime=01:30:00
#PBS -j oe

cd $PBS_O_WORKDIR

SRC="../../src/lb.cpp"
BIN="../../lb"
BIN_MPE="../../lb_mpe"

echo "=====
echo "   Сборка"
echo "=====

mpicc -mt_mpi -O2 -std=c++11 -o $BIN $SRC -lm -lpthread

# MPE: пробуем разные варианты линковки
MPE_OK=0
if command -v mpecxx >/dev/null 2>&1; then
    mpecxx -mpilog -mt_mpi -O2 -std=c++11 -o $BIN_MPE $SRC -lm -lpthread &&
    MPE_OK=1
fi
if [ $MPE_OK -eq 0 ]; then
    # ищем libmpe вручную
    MPE_LIB=$(find /usr /opt 2>/dev/null -name "libmpe.so" -o -name
"libmpe.a" 2>/dev/null | head -1)
    MPE_DIR=$(dirname "$MPE_LIB" 2>/dev/null)
    if [ -n "$MPE_DIR" ]; then
        mpicc -mt_mpi -O2 -std=c++11 -o $BIN_MPE $SRC -lm -lpthread \
            -L"$MPE_DIR" -lmpe -llmpe && MPE_OK=1
    fi
fi
if [ $MPE_OK -eq 0 ]; then
    echo "ВНИМАНИЕ: MPE недоступен, lb_mpe не собран"
    BIN_MPE=""
fi

echo "Сборка завершена."
echo ""

echo "=====
echo "   Замеры производительности"
echo "=====

NP_LIST="1 2 4 8 12 16 24"

for NP in $NP_LIST; do
    echo ""
    echo "--- NP = ${NP} ---"
    if [ "$NP" -le 12 ]; then
        mpirun -machinefile $PBS_NODEFILE -np $NP \
            $BIN > result_np${NP}.txt 2>&1
    else
        mpirun -machinefile $PBS_NODEFILE -np $NP -perhost 12 \
            $BIN > result_np${NP}.txt 2>&1
    fi
    grep "Tmax\\|imbalance" result_np${NP}.txt | head -10
done

echo ""
```

```

echo "=====
echo "   Сводная таблица"
echo "=====

T_BASE=$(grep "Tmax" result_np1.txt | awk '{sum += $3} END {printf "%.4f",
sum}')

printf "%-5s | %-14s | %-10s | %-10s\n" "NP" "T_total (s)" "S" "E"
printf -- "-----|-----|-----|-----\n"

for NP in $NP_LIST; do
    TT=$(grep "Tmax" result_np${NP}.txt | awk '{sum += $3} END {printf
"% .4f", sum}')
    if [ -n "$TT" ] && [ -n "$T_BASE" ] && [ "$T_BASE" != "0" ]; then
        S=$(awk "BEGIN{printf \"%.4f\", ${T_BASE}/${TT}}")
        E=$(awk "BEGIN{printf \"%.4f\", ${S}/${NP}}")
    else
        S="-"; E="-"
    fi
    printf "%-5s | %-14s | %-10s | %-10s\n" "$NP" "${TT:--}" "$S" "$E"
done

echo ""
echo "=====
echo "   МРЕ-профилировка, NP=16"
echo "=====

if [ -n "$BIN_MPE" ] && [ -f "$BIN_MPE" ]; then
    mpirun -machinefile $PBS_NODEFILE -np 16 -perhost 12 \
        $BIN_MPE > profile_np16.txt 2>&1
    grep "Tmax\\|imbalance" profile_np16.txt | head -10

    # clog2 может называться по имени бинарника
    if ls *.clog2 1>/dev/null 2>&1; then
        mv *.clog2 profile_np16.clog2
        echo "Лог МРЕ сохранён : profile_np16.clog2"
        echo "Открыть: jumpshot profile_np16.clog2"
    else
        echo "ВНИМАНИЕ: .clog2 не создан"
        echo "Проверьте: ldd $BIN_MPE | grep mpe"
        ls -la *.slog2 *.log 2>/dev/null || true
    fi
else
    echo "lb_mpe не собран, профилировка пропущена"
fi

echo ""
echo "=== Готово ==="

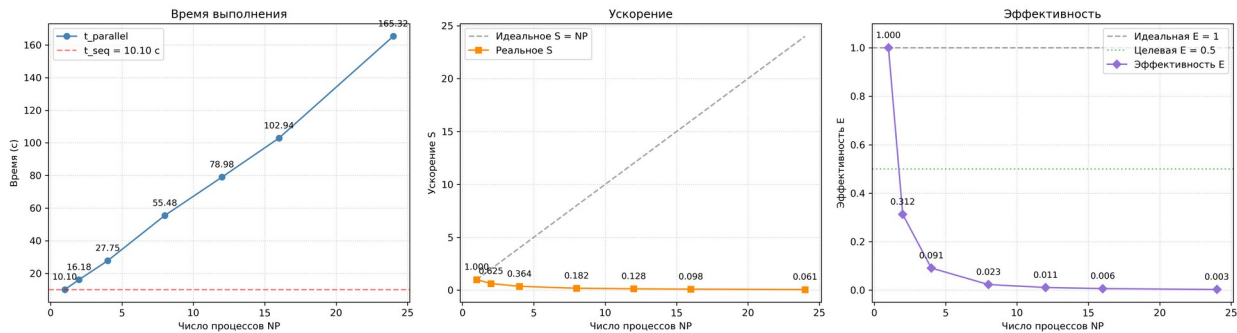
```

Приложение 3. Таблицы

№	ВРЕМЯ	УСКОРЕНИЕ	ЭФФЕКТИВНОСТЬ	ВРЕМЯ ПОСЛЕДОВАТЕЛЬНОЙ: 10.1037
2	16.1776	0.6245	31.23%	
4	27.7508	0.3641	9.10%	
8	55.4816	0.1821	2.28%	
12	78.9756	0.1279	1.07%	
16	102.9394	0.0982	0.61%	
24	165.3225	0.0611	0.25%	

Приложение 4. Графики

Динамическая балансировка нагрузки (Work Stealing)
MPI + pthreads, T_seq = 10.10 с



Приложение 5. Профилирование

